

Milestone 3: The Autonomous Shopping Cart

Tyler Potsiadlo, Tianxiang Liu, and Vedarsh Mishra
CSCI3387.01 Topics in Computational Intelligence

April 17, 2026

1 Evolution from Milestone 1

Based on feedback and feasibility analysis, several core concepts from Milestone 1 have been updated:

- **Scope Shift (1D Reinforcement Learning to Jerkiness Control):** We determined that implementing Reinforcement Learning (RL) to fully control the 1D straight-line case end-to-end is redundant, as there are simpler deterministic control laws to achieve this. Instead, we have shifted the RL agent to act as a residual smoothing layer. For example, if a shopper comes to an abrupt stop, a deterministic algorithm may fire a sharp velocity-zero command, causing an extremely fast, unstable deceleration. By incentivizing the RL model to ensure changes in velocity are gradual, the cart comes to a smooth stop and carefully backs up to the optimal position, keeping the shopper’s items safe.
- **Turn-in-Place Mechanics:** We are engineering the cart to support zero-radius turns (turning in place). This removes the need for complex, hard-coded non-holonomic kinematics (like Clothoid S-curves) and heavily simplifies the spatial reasoning required in the aisles.

2 Introduction

The problem we are addressing sits within the broader context of autonomous vehicles and assistive robotics: hands-free cargo transport in pedestrian environments. Specifically, we are designing an autonomous front-leading shopping cart that navigates ahead of a user, allowing them to browse shelves and place items into the cart without manually pushing it.

Following a user from the front—rather than trailing behind—presents unique control and prediction challenges. The cart must continuously estimate the user’s intended trajectory using posture cues (e.g., shoulder rotation) and execute maneuvers to change lanes without colliding with shelves. Furthermore, safety and collision avoidance are paramount; the system must gracefully handle dynamic obstacles and user unpredictability.

3 Toy Problem: Mock Examples and Edge Cases

To successfully operate in a grocery store, the cart must fluidly switch between different behavioral regimes.

- **Good Behavior - 1D Steady-State Following:** The cart maintains a strict 0.5m to 0.8m leading distance ahead of the user while traveling down a straight aisle, smoothly matching the user’s velocity without oscillating.

- **Good Behavior - Cornering and Lane Changes:** When the user turns their shoulders to enter a new aisle, the cart anticipates the turn. Because it is engineered to turn in place, it rotates to match the user’s new heading and re-establishes its 1D tracking trajectory, drastically reducing the required turn radius.
- **Bad Behavior - Dead Reckoning Drift:** The cart executes a turn but fails to use closed-loop visual correction, permanently drifting at an incorrect angle and eventually colliding with an aisle shelf.
- **Collision Avoidance (Edge Case):** If a shopper steps between the cart and the user, the cart must immediately trigger an emergency stop. The cart will not attempt to autonomously navigate around the obstacle; instead, it will remain locked until the user manually resets it.

4 Related Work

Our architecture builds upon prior research in human-following robotics and motion prediction:

- **Rule-Based Smart Carts:** Sanghavi et al. [1] developed a human-following smart cart utilizing ultrasonic sensors and magnetometers. While effective for basic trailing, their purely reactive rule-based approach struggles with the predictive requirements of a front-leading cart.
- **Predictive Leading Navigation:** Wang [2] demonstrated that predicting human walking motion even a short time into the future significantly improves tracking performance. We incorporate this by tracking shoulder vectors for early turn detection.
- **RNNs for Human Trajectory:** Moon & Seo [3] explored using Recurrent Neural Networks to predict human trajectories. While we use Deep Reinforcement Learning for longitudinal control, their insights into biomechanical sway and gait noise inform our filtered observation space.

5 Data Sources

5.1 Description of the Data

- **Video Data:** We use a single monocular RGB camera mounted on the cart’s rear-facing side to record video of the person being followed (30 FPS at 2304×1296 resolution). Data is collected in indoor hallways and open rooms with varied lighting.
- **Ground Truth Relative Position:** We collect ground truth distance and horizontal angle using an ArUco marker and OpenCV’s `cv2.aruco` library. The person wears a printed marker, allowing us to accurately measure the 3D relative position via `estimatePoseSingleMarkers`.
- **Motor Commands:** We log the commanded throttle and steering values sent to the cart at each step, synchronized with the camera frames. This allows us to train the reinforcement learning policy on exact state-action trajectories.

5.2 Why the Data is Optimal

Our dataset directly provides the exact quantities the control system needs. The orientation and turn intent do not come from the ArUco marker; rather, YOLOv8 Pose keypoints (shoulders and

hips) provide the biomechanical cues mathematically necessary to predict turns before the trajectory changes. The horizontal angle of the person is derived geometrically from the bounding-box center and the camera’s focal length, giving the RL agent a polar-coordinate observation (r, θ) .

Note on Accuracy: The ”sub-centimeter” accuracy claim for ArUco markers is cited from existing literature (Ošćádal et al. [5]), which reports millimeter-scale translation error under favorable conditions (marker size in pixels, calibration quality, and corner refinement). We are currently validating this on our own hardware with tape-measured static tests over a 0.5–3m operating range.

5.3 How the Data Will Be Used (The Decoupling Strategy)

To prevent the control logic from bottlenecking behind the perception module, we have decoupled the pipeline:

1. **Phase 1 (Current):** The person wears an ArUco marker, providing direct distance and angle ground truth to supervise the RL agent’s training entirely independent of the YOLO model’s completion.
2. **Phase 2:** We will replace the marker with a lightweight learned model that takes YOLOv8-Pose shoulder-width and torso-length features to regress the same $(distance, angle)$ pair.

This data is utilized synergistically with our custom **Vanguard2D** Gymnasium environment. The RL model directly uses the computer vision information (distance) as a core part of its input observation space, bridging the synthetic and real-world inputs.

6 Hardware Platform

The physical prototype is built on a compact RC chassis approximately 7 inches (18cm) tall, running a Raspberry Pi 3B as the onboard compute unit. The Pi Camera is connected via the CSI port and validated using `libcamera-hello`. Motor control is handled by two continuous rotation servos: the left-side wheels are wired together on GPIO 18 (Physical Pin 12) and the right-side wheels on GPIO 13 (Physical Pin 33), controlled via 50 Hz PWM.

6.1 Hardware Debugging & Iteration

Hardware integration involved several non-trivial debugging cycles across multiple Raspberry Pi units:

- **Overheating:** Under combined camera and GPIO load, the Pi thermally throttled and dropped SSH connections. Resolved via a two-step fix: adding a heatsink to the CPU, then mounting an active cooling pad, bringing operating temperatures into a stable range.
- **Network Connectivity:** The BC campus network blocks peer-to-peer SSH connections required for headless operation. The Pi connected to WiFi successfully but was unreachable via SSH. Resolved by switching to a personal hotspot, which is now standard procedure for all hardware sessions.
- **Power Architecture:** Initially both servos and camera were powered from the Pi’s 5V GPIO pins, causing the Pi to brown out. Resolved by separating power sources: the Pi runs on its own dedicated supply and the servos are powered independently. This also eliminated a class of servo jitter that was initially misdiagnosed as a software issue.

- **Servo Calibration & Jitter:** After resolving power, residual servo issues remained — incorrect PWM neutral pulse width requiring trimming, `RPi.GPIO` library setup, and jitter at the stop position traced to PWM timing instability at low duty cycles. Resolved by tuning the stop duty cycle constant in the motor driver.

6.2 Control Dashboard

To make the hardware accessible to all team members without requiring physical access to the cart, a web-based control dashboard was built. The backend connects to the Pi over SSH — the moment the cart powers on and joins the hotspot, it auto-connects. From the dashboard, any team member can view a live camera feed, capture recordings for CV training data, control and trim motor commands, and remap GPIO pins remotely. This decouples hardware access from physical presence, which is critical given that only one team member has the cart.

6.3 Prototype Form Factor & Camera Height

The current prototype is significantly smaller than a real shopping cart, which stands approximately 90–100 cm tall. We acknowledge that the extreme upward viewing angle introduces distortion in keypoint geometry, potentially reducing shoulder rotation inference accuracy at close range. One solution being actively explored is a rotating camera head that can tilt to compensate for the low mount height. The RC chassis is intentionally a low-cost platform for validating the ML and control pipeline before committing to a larger form factor.

7 Machine Learning Methods

To solve the problem, we blend deep reinforcement learning for dynamic smoothing with classic robotics control theory for baseline navigation.

7.1 Reinforcement Learning: Proximal Policy Optimization (PPO)

Algorithm Details & Context: In this architecture, Reinforcement Learning acts strictly as a residual smoothing layer. The deterministic controller dictates the raw target speed, and the RL agent determines the necessary adjustments to ensure the cart achieves these speeds without jerking.

We utilize PPO, an **Actor-Critic** policy gradient method. In the context of our cart:

- **The Actor:** This is the neural network that evaluates the current observation and outputs the actual physical command. It decides exactly how much velocity to add or subtract to the cart’s baseline target.
- **The Critic:** This is a secondary neural network that evaluates the “goodness” of a given state. It looks at the cart’s current situation (e.g., driving 1.0 m/s while 1.5 meters away from the user) and predicts the expected future reward, acting as a baseline to tell the Actor if its chosen action was better or worse than expected (the “Advantage”).

The model does not operate as a state machine; rather, it outputs a fresh prediction at every timestamp. PPO optimizes a clipped surrogate objective function:

$$L^{CLIP}(\theta) = \hat{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (1)$$

Optimization Breakdown: To maximize this objective function, PPO utilizes **Stochastic Gradient Ascent** to iteratively adjust the Actor’s neural weights over multiple epochs using sampled batches of trajectory data. The function calculates its updates using the following core mathematical components:

- **The Probability Ratio ($r_t(\theta)$):** This ratio measures the probability of the cart taking a specific action under the *new* updated policy compared to the *old* policy.
- **The Advantage ($\hat{A}_t$):** Computed by the Critic network, this value represents how much better (or worse) a specific velocity adjustment was compared to the baseline expectation.
- **The Clipping Mechanism:** PPO multiplies the probability ratio by the Advantage to encourage the Actor to repeat successful velocity adjustments. However, the `clip` function rigidly restricts this probability ratio within a tight mathematical boundary (typically $[1 - \epsilon, 1 + \epsilon]$, where $\epsilon \approx 0.2$).
- **The Min Operator:** By taking the minimum between the unclipped and clipped terms, the algorithm actively truncates the gradient. If the cart performs a "lucky" adjustment that yields a massive positive Advantage, the unclipped ratio would normally cause the network weights to violently spike in that direction. The Min operator forces the gradient to zero beyond the $1 + \epsilon$ bound, ignoring the spike.

This conservative optimization strategy prevents the network from taking destructively large step sizes (which could instantly destabilize the cart’s physical driving policy and lead to unrecoverable "catastrophic forgetting") and ensures the prototype smoothly and monotonically converges toward jerk-free tracking.

State, Action Space, and Execution: The environment provides the RL agent with state feedback: the cart’s current distance from the user (via CV), current velocity, and the baseline deterministic velocity. The action space is continuous, defined as a modifier $a_t \in [-1, 1]$, representing the proportion of maximum velocity to add to or subtract from the deterministic base (e.g., modifying the base speed by -0.2 m/s).

Reward Function: The reward function evaluates how close the cart is to the target gap using a Euclidean distance penalty (e.g., if the target is 1.2m and the actual is 1.6m, the penalty scales with the 0.4m error). Crucially, to penalize jerkiness, the reward function heavily subtracts points for abrupt changes in velocity between steps. Thus, PPO is incentivized to maintain the target distance while actively flattening the acceleration curve.

7.2 Computer Vision: YOLOv8 Pose Estimation

Algorithm Details: We use a pretrained YOLOv8 Pose Nano model as the frozen backbone of our perception pipeline. This single-stage detector produces each person’s bounding box and 17 COCO body keypoints in one forward pass. On top of the frozen backbone, we train lightweight heads to extract distance (supervised by ArUco ground truth), horizontal angle, and a turn-intent classifier over a temporal window of shoulder/hip keypoints.

Why Selected: YOLOv8 Pose Nano has only 3.3M parameters, avoiding the latency of chaining a separate detector and pose network.

Implementation: Each frame is passed through YOLOv8 Pose. The extracted distance and angle form a polar-coordinate observation (Z, α) that is passed directly to the deterministic controller and RL agent.

7.3 Deterministic 1D Control

Because we are engineering the cart to turn in place, we have eliminated the need for hard-coded non-holonomic kinematics. This allows us to treat the primary following task strictly as a 1D longitudinal problem using a mathematical Proportional (P) tracking algorithm.

This algorithm acts as an invisible, elastic tether between the cart and the user. On every frame, the controller calculates the exact velocity required to maintain the gap using the following mathematical logic:

1. **Error Calculation:** The system measures the difference between the physical distance and the desired gap.

$$E_t = D_{actual} - D_{target} \quad (2)$$

2. **Proportional Gain & Feed-Forward:** The error is multiplied by a tuning constant (K_p) which acts as the "stiffness" of the tether. Because this is a front-leading Vanguard cart, we must also add the user's current velocity (V_{person}) as a feed-forward term so the cart paces the user rather than purely waiting for an error to occur.

$$V_{base} = -K_p(E_t) + V_{person} \quad (3)$$

3. **Safety Clamping:** Finally, the raw mathematical command is clamped to ensure it does not exceed the physical capability or safety limits of the cart's motor driver (V_{max}).

$$V_{cmd} = \max(-V_{max}, \min(V_{max}, V_{base})) \quad (4)$$

Once this hard-coded V_{cmd} is established, it is passed to the RL agent, which outputs its residual action to smooth the final acceleration curve. When a turn is detected, the cart overrides this 1D logic, utilizing differential thrust on its wheels to rotate in place and re-align with the user's new heading before resuming longitudinal tracking.

8 Success-Assessment Plan

We will evaluate the system's success across simulated and real-world metrics:

1. **Longitudinal Error (RL Performance):** Measured by the average variance from the target 0.5m leading gap during steady-state straightaways.
2. **Jerk Reduction:** Measured by the peak acceleration/deceleration gradients compared to a purely deterministic baseline.
3. **Collision Rate:** The frequency at which the cart allows the gap to exceed 3.0m or drop below 0.3m.
4. **Vision Metrics:** The perception pipeline must run at ≥ 10 FPS end-to-end. Distance estimation error must be ≤ 0.1 m (MAE) over the 0.5–3m operating range, validated against ArUco ground truth.

To fulfill the final course deliverable, we will provide a live Google Colab document containing the trained RL inference environment that runs out-of-the-box.

9 Plan and Strategy

- **Task 1: Simulation & RL Training (Tyler)** - Update the training environment to reflect recently-changed RL goals. Experiment with hyperparameter choices in RL training to ensure a healthy mix between smoothness and maintaining correct distance.
- **Task 2: Vision & Perception (Jimmy)** - Build the Phase 2 learned head on top of YOLOv8 to extract depth and shoulder heading from live camera feeds.
- **Task 3: Hardware Integration (Vedarsh)** - Integrate the compute board, depth camera, motor controllers and SSH dashboard. Ensure the differential drive can execute the turn-in-place logic.
- **Task 4: System Merge & Testing (All)** - Use pre-trained PPO weights to train model cart to operate in real environment. This will involve feeding the CV observations into the RL model, fetching commands, and updating in real-time.

10 Contributions

Day 1 to Milestone 1:

- **Tyler:** Drafted initial problem statements. Researched reinforcement learning candidates, explicitly comparing DDPG, SAC, and PPO. Ruled out DDPG due to its high hyperparameter sensitivity in continuous control tasks. While SAC was initially tested for its sample efficiency, it was ultimately ruled out due to its core mechanic of entropy maximization. SAC actively encourages the agent to explore highly randomized, high-variance actions to maximize entropy alongside reward. In a physical, continuous robotic control task like maintaining a precise target gap, this entropy-driven exploration led to erratic, jittery throttle commands that actively violated our goal of jerk reduction. PPO was firmly selected for its clipped on-policy updates, providing much safer, smoother convergence for the physical cart prototype.
- **Jimmy:** Explored CV tracking frameworks and drafted the CV training strategy section.
- **Vedarsh:** Drafted hardware requirements, researched RC car platforms, and helped structure the evaluation matrix for project selection.

Milestone 1 to Milestone 2:

- **Tyler:** Developed a custom `gym.Env` in Python. Refactored the control schema from a purely deterministic setup to a hybrid residual RL system. Designed the high-degree polynomial reward function to penalize jerkiness and ran initial PPO training loops using Stable Baselines3.
- **Jimmy:** Designed the full computer vision pipeline architecture. Selected YOLOv8 Pose Nano as the frozen backbone, defined the geometric computation for distance/angle estimation, and established the Phase 1 (ArUco) to Phase 2 (Learned Head) decoupling strategy to unblock the RL training pipeline.
- **Vedarsh:** Set up the Raspberry Pi headless development workflow. Evaluated multiple capturing backends (`picamera2` vs `opencv`) and ultimately deployed a Uvicorn HTTP streaming server to expose the camera feed over the local network, circumventing local hardware limitations and establishing the motor-control API contract.

Milestone 2 to Milestone 3:

- **Tyler:** Reformulated problem to simplify RL task. Selected algorithm for deterministic control (before RL smoothing) in 1D line case. Set up integration of computer vision via connectivity to car onto own computer. Research more related works and contribute further to references section to make it richer. Didn't have sufficient time to do this before this milestone, but making an important note of it to improve upon this for final submission.
 - **Jimmy:** Restored high-FPS live streaming. Added Record and RL Training tabs to the web dashboard. Run an ArUco and YOLO pose accuracy test over the 0.5–3m range.
 - **Vedarsh:** Resolved Pi overheating, BC network SSH blocking, and servo brownout via dual power sourcing. Calibrated PWM timing and eliminated servo jitter. Built a web-based SSH dashboard for live camera, motor control, and data capture.
-

References

- [1] S. Sanghavi, P. Rathod, P. Shah, and N. Shekokar, "Design and Implementation of a Human Following Smart Cart," *Proceedings of the Third International Conference on Smart Systems and Inventive Technology (ICSSIT)*, IEEE, 2020.
- [2] A. Wang, "Human Motion Prediction Applicable to Human-Following Robot," *Doctoral Thesis*, 2022.
- [3] H.-S. Moon and J. Seo, "Prediction of Human Trajectory Following a Haptic Robotic Guide Using Recurrent Neural Networks," *Proceedings of the IEEE World Haptics Conference (WHC)*, pp. 157-162, 2019.
- [4] Google, "Gemini," Large language model used for document formatting and structural organization, 2026. [Online]. Available: <https://gemini.google.com>
- [5] P. Oščádal et al., "Improved Pose Estimation of Aruco Tags Using a Novel 3D Placement Strategy," *Sensors*, vol. 20, no. 17, p. 4825, 2020. doi:10.3390/s20174825.
- [6] OpenCV, "Detection of ArUco Markers," OpenCV 4.x tutorial. https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html